



Programmation Scripts Intégrés

SED-1515 A00

Projet Groupe - I

Conception de haut niveau et Calendrier

Groupe 3

Kelly-Ann Lessard 300307532

Ange Yvan Mugisha 300505715

Mufuta Butoka David 300383643

David Berosy 300511667

Présenté à : Yassine Hammadi

TA : AudreyAnn Bleau-Brousseau

Université d'Ottawa

Date de soumission : 2 novembre 2025

Table des Matières

Introduction.....	3
1. Objectif du Programme.....	4
2. Logiciel et Outils.....	4
2.1. VS Code.....	4
2.2. Outils.....	4
2.2.1. Git & Github.....	4
2.2.2. Copilot.....	4
3. Lists des Modules.....	5
3.1. Modules à écrire.....	5
3.2. Module Python Utilises.....	5
4. Conception.....	6
Tableau #1: Conception de Code.....	6
Tableau #2: Conception de Câblage.....	6
5. Structure de Données.....	6
5.1. Types de données utilisés.....	7
5.1.1. Listes:.....	7
5.1.2. Dictionnaires:.....	7
5.1.3. Booléens:.....	7
5.2. Organisation logique.....	7
6. Test Possibles.....	8
7. Calendrier.....	9
Photo #1: Calendrier Notion Page Principale.....	9
Photo #2: Tâches de Projet.....	10
Référence.....	11

Introduction

Ce document présente la planification de haut niveau pour le développement d'un système embarqué permettant à un microcontrôleur Raspberry Pi Pico de contrôler un brachiographe à trois servomoteurs. Ce projet s'inscrit dans le cadre du cours SED1515 – Conception de logiciels embarqués et écriture de scripts à l'Université d'Ottawa.

Ce rapport décrit :

- Le plan de développement par étapes, incluant les tâches de conception, de codage et de test.
- Le schéma fonctionnel de l'architecture logicielle.
- La liste des outils et modules logiciels utilisés ou réutilisés, avec les licences associées.
- Le flux d'outils et environnement de programmation/test.
- Le tableau des scénarios de test fonctionnel, avec estimation du temps et de la complexité.

Ce projet met l'accent sur la modularité, la collaboration via GitHub, et la réutilisation raisonnée de logiciels open source, tout en respectant les contraintes mécaniques et les limites du matériel embarqué.

1. Objectif du Programme

L'objectif principal est de concevoir un programme en Python capable de piloter en temps réel les mouvements d'un bras traceur à l'aide de deux potentiomètres analogiques (axes X et Y) et d'un interrupteur pour la commande du stylo (haut/bas). Le système doit interpréter les entrées analogiques, les transformer en angles articulaires, et générer les signaux PWM nécessaires pour actionner les servomoteurs de l'épaule, du coude et du stylo.

2. Logiciel et Outils

2.1. VS Code

Environnement de développement principal Extensions utiles :

- Pico-Go pour flasher et interagir avec le Raspberry Pi Pico
- Python pour l'autocomplétion et le linting
- GitHub pour la gestion de version intégrée

Avantages : interface puissante, intégration Git, personnalisation, support multi-fichiers

2.2. Outils

2.2.1. Git & Github

- Hébergement du code source Suivi des versions et des modifications
- Utilisation de GitHub Issues pour suivre les bogues, les tâches et les jalons
- Branches par fonctionnalité ou membre de l'équipe

2.2.2. Copilot

Dans le cadre de ce projet, Copilot sera utilisé comme un outil d'assistance intelligent pour soutenir la planification, la conception, le codage et la documentation du système embarqué. Grâce à ses capacités de traitement du langage naturel et de recherche contextuelle, Copilot permet de :

- Clarifier les exigences techniques à partir des documents fournis (comme les consignes du projet).
- Générer du code Python adapté au Raspberry Pi Pico, incluant la gestion des signaux PWM, la lecture des entrées analogiques et la transformation inverse cinématique.
- Structurer le rapport technique, en proposant des introductions, des sections organisées, des tableaux de tests et des schémas fonctionnels.
- Optimiser le flux de travail, en suggérant des outils, des modules logiciels, et des méthodes de test reproductibles.
- Faciliter la collaboration, en aidant à rédiger des descriptions claires pour GitHub Issues, des commentaires de code, et des guides d'utilisation pour les membres de l'équipe.

Copilot agit comme un assistant technique et rédactionnel, capable de s'adapter aux besoins du projet en temps réel, tout en renforçant la rigueur, la clarté et la reproductibilité des livrables.

3. Lists des Modules

3.1. Modules à écrire

- Lecture analogique et filtrage Transformation inverse cinématique
- Gestion du stylo (haut/bas)
- Générateur PWM
- Interface utilisateur pour le débogage

3.2. Module Python Utilises

- Machine
- Time / Uptime
- Math

4. Conception

Tableau #1: Conception de Code

brachiographe/	
— main.py	# Point d'entrée du programme
— adc_reader.py	# Lecture des potentiomètres X et Y
— inverse_kinematics.py	# Calcul des angles pour épaule et coude
— pwm_controller.py	# Commande des servos via PWM
— pen_control.py	# Gestion du stylo (haut/bas)
— utils.py	# Fonctions utilitaires (filtrage, conversion)

Tableau #2: Conception de Câblage [2]

```
from machine import Pin, ADC, PWM

# Potentiomètre X branché sur GP26 (ADC0)
pot_x = ADC(26)

# Potentiomètre Y branché sur GP27 (ADC1)
pot_y = ADC(27)

# Interrupteur branché sur GP16, avec résistance de tirage interne activée
switch = Pin(16, Pin.IN, Pin.PULL_UP)

# Servo de l'épaule sur GP15
servo_epaule = PWM(Pin(15))

# Servo du coude sur GP14
servo_coude = PWM(Pin(14))

# Servo du stylo sur GP13
servo_stylo = PWM(Pin(13))
```

5. Structure de Données

La conception logicielle du brachiographe repose sur des structures de données simples et efficaces, permettant de gérer les entrées analogiques, les transformations mathématiques, et les commandes des servomoteurs de manière modulaire et lisible.

5.1. Types de données utilisés

5.1.1. Listes:

- Pour stocker les valeurs successives des potentiomètres (X, Y) lors du filtrage ou de la moyenne glissante.
- Pour enregistrer les coordonnées de traçage si une fonction de mémoire ou de dessin est ajoutée.

5.1.2. Dictionnaires:

- Pour associer chaque servomoteur à ses paramètres PWM (broche, fréquence, rapport cyclique).
- Pour organiser les angles calculés par articulation
({"epaule": angle1, "coude": angle2})

5.1.3. Booléens:

- Pour représenter l'état du stylo (haut ou bas) selon l'entrée de l'interrupteur.
- Nombres flottants (**float**) :
- Pour les valeurs analogiques lues (entre 0.0 et 3.3V).
- Pour les angles calculés en degré.
- Pour les coordonnées X/Y et les longueurs des segments du bras.

5.2. Organisation logique

Chaque module logiciel manipule ses propres structures de données :

- Le module ADC lit les tensions et les convertit en valeurs numériques.
- Le module cinématique inverse transforme les coordonnées X/Y en angles articulaires.
- Le module PWM utilise ces angles pour générer les signaux de commande des servos.
- Le module stylo gère l'état haut/bas selon l'interrupteur.

Cette organisation favorise la modularité, la lisibilité du code, et la facilité de test. Elle permet aussi une extension future vers des fonctions plus avancées comme le dessin automatique ou la mémorisation des trajectoires.

6. Test Possibles

La fiabilité du système embarqué repose sur une série de tests fonctionnels rigoureux, visant à valider chaque composant logiciel et matériel du brachiographe. Ces tests permettront de s'assurer que les entrées analogiques sont correctement interprétées, que les transformations mathématiques sont précises, et que les servomoteurs réagissent de manière cohérente aux commandes utilisateur.

Les tests seront réalisés en plusieurs étapes, en suivant une logique de validation progressive :

- D'abord, tester chaque module individuellement (lecture analogique, transformation inverse, commande PWM).
- Ensuite, tester les interactions entre modules.
- Enfin, effectuer des tests d'intégration sur le système complet avec le bras mécanique.

Chaque test sera documenté dans un tableau détaillant :

- Le nom du test
- La fonctionnalité ciblée
- Le type d'entrée simulée ou réelle
- Le comportement attendu
- Le temps estimé pour l'écriture, l'exécution et la correction
- Le niveau de difficulté (simple, modéré, complexe)

Les tests seront réalisés manuellement à l'aide du matériel fourni, et complétés par des scripts de vérification lorsque possible. Les résultats seront enregistrés pour faciliter le débogage et l'amélioration continue du système.

Cette approche garantit une traçabilité complète, une réduction des risques d'erreur, et une meilleure reproductibilité des performances du traceur.

7. Calendrier

Pour ce projet nous avons utilisé l'application Notion pour le calendrier de projet et le dressage et répartition des tâches. Voici le [lien Notion](#). [2]

SED1515 Projet Groupe 3

Code du cours : SED1515

Professeur :
Nom : Yassine Hammadi
Courriel : yhammadi@uottawa.ca

TA:
Nom : AudreyAnn
Courriel : ablea057@uottawa

Plan du projet

Tâches de projet

Table Gantt Calendar

Status Nom de tâche Date d'échéance Jalons Dependant sur Priorité + ...

✓	Mise à jour Notion	October 9, 2025 → November 7, 2025	Tâche		Bas	
✓	Déterminer l'Objectif	October 30, 2025 23:59	Tâche		Moyen	
✓	Calendrier Notion	October 31, 2025 23:59	Tâche		Moyen	
✓	Créer un Plan de Conception	October 31, 2025 23:59	Tâche	Déterminer l'Objectif	Moyen	
✓	Livrable A	November 2, 2025 23:59	Livrable	Calendrier Notion Créer un Plan de Conception	Haut	

Photo #1: Calendrier Notion Page Principale

Référence

[1] N. Panchal, "Raspberry Pi Pico Servo Motor Control - Electric DIY Lab," *Electric DIY Lab*, Apr. 02, 2021.

https://electricdiylab.com/raspberry-pi-pico-servo-motor-control/?utm_source=chatgpt.com
(accessed Nov. 02, 2025).

[2] "The AI workspace that works for you. | Notion," *Notion*, 2025.

https://www.notion.so/SED1515-Projet-Groupe-3-26eb170387ad81a1b932f72c58271b27?source=copy_link (accessed Nov. 02, 2025).